TASK 4
IoT CLOUD INTEGRATION & REAL-TIME REMOTE MONITORING

Internship Project Report

Student Name	Yuvashri
Institution	Dayananda Sagar University
Internship Organization	Maincrafts Technology
Platform	Wokwi ESP32 Simulator and ThingSpeak

1. INTRODUCTION

This task upgrades the Smart Temperature Monitoring and Automation System into a cloud-connected IoT application. An ESP32 reads temperature from an NTC sensor, displays the reading and status on an LCD, controls three LEDs according to temperature thresholds, and sends readings through Wi-Fi to ThingSpeak for remote monitoring and graph visualization.

2. OBJECTIVE

To develop a working cloud-connected temperature monitoring system and demonstrate sensor acquisition, local automation, Wi-Fi connectivity, cloud data publishing, and remote visualization.

3. HARDWARE AND SOFTWARE REQUIREMENTS

- ESP32 DevKit V1 – microcontroller with Wi-Fi and ADC input
- NTC temperature sensor – temperature measurement in Wokwi
- LCD1602 I2C – local display of temperature and status
- Green, yellow and red LEDs with 220 Ω resistors – status indication
- Wokwi – online circuit simulator and ESP32 development environment
- ThingSpeak – cloud channel, data storage and temperature chart

4. SYSTEM ARCHITECTURE AND WORKING PRINCIPLE

The NTC sensor provides an analog signal to ESP32 GPIO34. The ESP32 reads the ADC value and calculates temperature using the thermistor Beta equation. The firmware selects a status and activates the corresponding LED. The LCD displays the temperature and status. At a 20-second interval, the ESP32 sends the temperature as Field 1 data to ThingSpeak using an HTTP request over Wi-Fi.

Data path: NTC sensor → ESP32 ADC and processing → LCD/status LEDs → Wi-Fi → ThingSpeak → temperature chart.

5. CIRCUIT CONNECTIONS

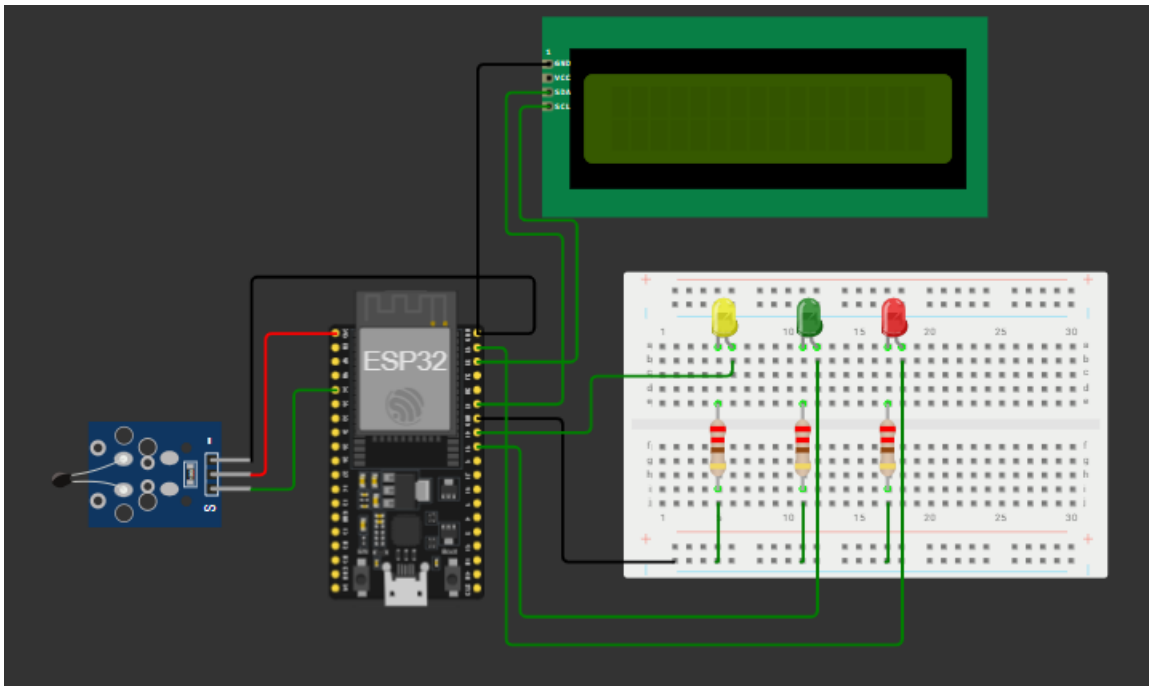


Fig 1: Complete ESP32-based Smart Temperature Monitoring System circuit in Wokwi.

Component pin	ESP32 connection
NTC VCC	3V3
NTC OUT	GPIO34
NTC GND	GND
LCD GND	GND
LCD VCC	3V3
LCD SDA	GPIO21
LCD SCL	GPIO22
Green LED	GPIO18 through 220 Ω resistor; cathode to GND
Yellow LED	GPIO19 through 220 Ω resistor; cathode to GND
Red LED	GPIO23 through 220 Ω resistor; cathode to GND

6. TEMPERATURE-BASED AUTOMATION

Temperature	Status	LED indication
Below 25 °C	NORMAL	Green ON
25 °C to 35 °C inclusive	WARNING	Yellow ON
Above 35 °C	CRITICAL	Red ON

7. THINGSPEAK CLOUD INTEGRATION

A private ThingSpeak channel named “Smart Temperature Monitoring” was created with Channel ID 3566929. Field 1 was configured for temperature. The ESP32 connects to the Wokwi-GUEST Wi-Fi network and submits the temperature value to the ThingSpeak update endpoint using an HTTP GET request and the channel Write API Key. The firmware uploads data every 20 seconds. The Write API Key is intentionally omitted from this report for security.

During testing, the Serial Monitor displayed “ThingSpeak response: 2” and “Temperature uploaded successfully!”. A positive numerical response indicates that ThingSpeak created a new channel entry.

8. TESTING AND RESULTS

Observed temperature	Status	Expected LED
24.00 °C	NORMAL	Green
28.31 °C	WARNING	Yellow
37.89 °C	CRITICAL	Red

The Serial Monitor displayed temperature readings and their corresponding status. The ThingSpeak Field 1 chart showed successive readings, beginning near 24 °C and later around 37–38 °C. This verified that the cloud received and plotted the temperature data.

```
----------------------------
----------------------------
Temperature: 68.97 C
Status: CRITICAL
----------------------------
----------------------------
Temperature: 30.71 C
Status: WARNING
----------------------------
----------------------------
Temperature: 30.71 C
Status: WARNING
----------------------------
----------------------------
Temperature: 13.70 C
Status: NORMAL
----------------------------
----------------------------
```

Figure 2: Serial Monitor displaying temperature readings and successful cloud data transmission.

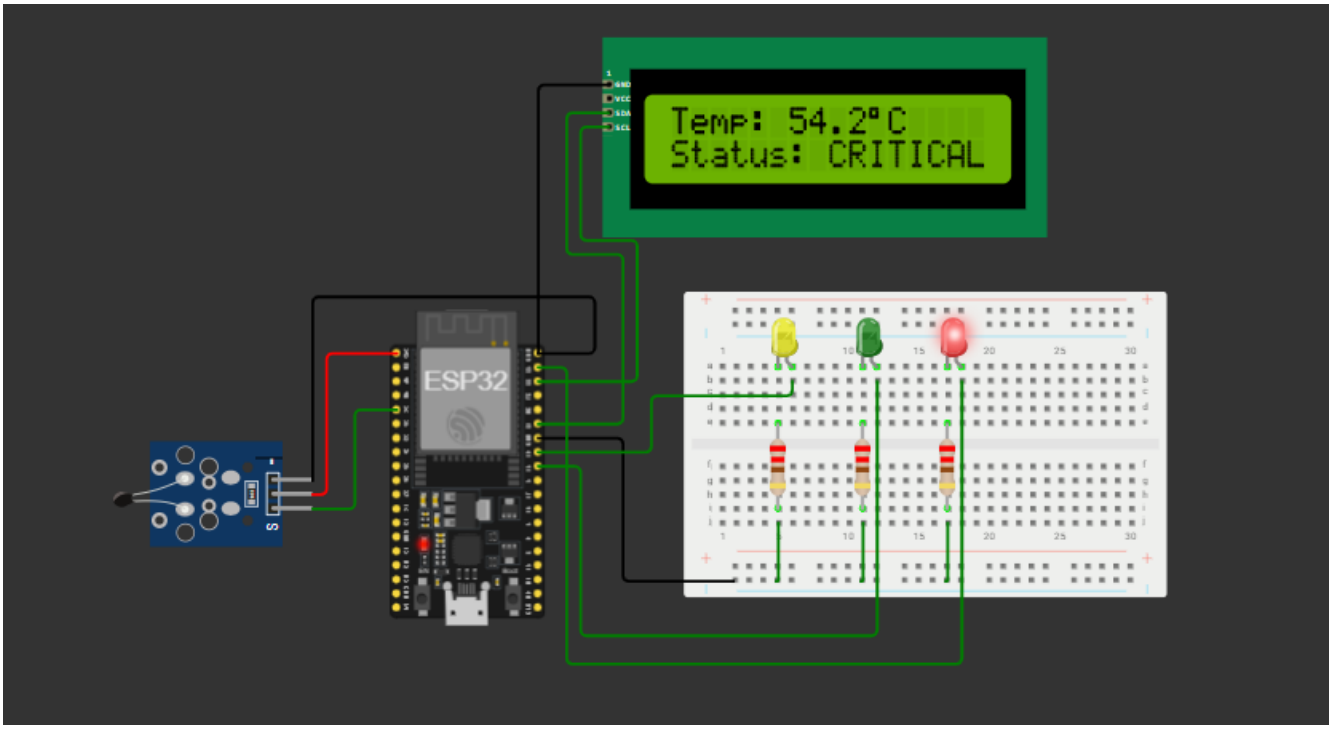


Figure 3: LCD-based real-time temperature and status monitoring.

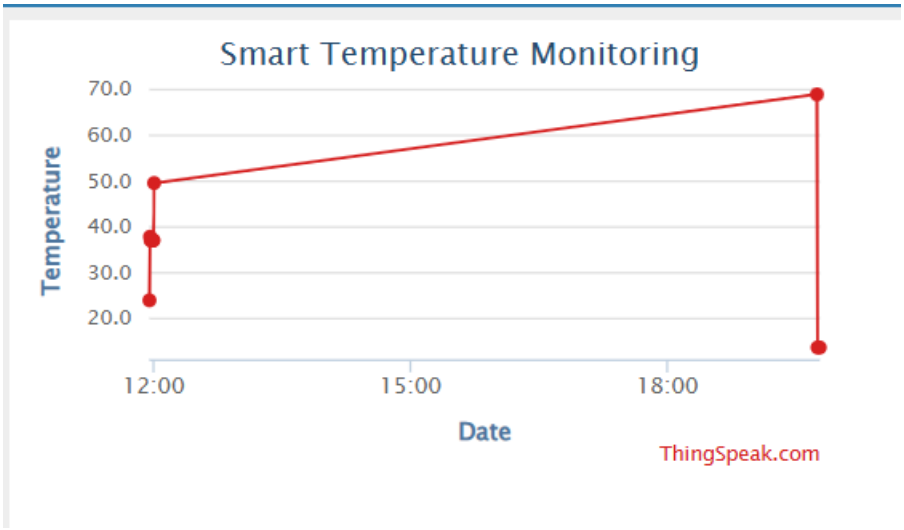


Figure 4: ThingSpeak cloud dashboard displaying real-time temperature data.

9. SOURCE CODE

The following code is the ESP32 Arduino sketch used for the simulation. Replace the placeholder with the current private ThingSpeak Write API Key in Wokwi. Do not include the actual key in a public report or screenshot.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```
#include <math.h>

const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

const char* THINGSPEAK_API_KEY = "YOUR_WRITE_API_KEY";
const char* THINGSPEAK_URL = "http://api.thingspeak.com/update";

#define NTC_PIN 34
#define GREEN_LED 18
#define YELLOW_LED 19
#define RED_LED 23

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define NORMAL_LIMIT 25.0
#define WARNING_LIMIT 35.0
#define BETA 3950.0
#define T0 298.15

unsigned long previousUpload = 0;
const unsigned long uploadInterval = 20000;

void connectWiFi()
{
  Serial.println("Connecting to Wi-Fi...");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\\nWi-Fi connected!");
  Serial.println(WiFi.localIP());
}

float readTemperature()
{
  int adcValue = analogRead(NTC_PIN);
  if (adcValue <= 0 || adcValue >= 4095) return NAN;

  float temperature = 1.0 /
    (log(1.0 / (4095.0 / adcValue - 1.0)) / BETA
    + 1.0 / T0) - 273.15;

  return temperature;
}

String controlLEDs(float temperature)
{
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
```

```
  digitalWrite(RED_LED, LOW);

  if (temperature < NORMAL_LIMIT)
  {
    digitalWrite(GREEN_LED, HIGH);
    return "NORMAL";
  }
  else if (temperature <= WARNING_LIMIT)
  {
    digitalWrite(YELLOW_LED, HIGH);
    return "WARNING";
  }
  else
  {
    digitalWrite(RED_LED, HIGH);
    return "CRITICAL";
  }
}

void displayLCD(float temperature, String status)
{
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C");
  lcd.setCursor(0, 1);
  lcd.print("Status: ");
  lcd.print(status);
}

void uploadToThingSpeak(float temperature)
{
  if (WiFi.status() == WL_CONNECTED)
  {
    HTTPClient http;
    String url = String(THINGSPEAK_URL)
              + "?api_key=" + THINGSPEAK_API_KEY
              + "&field1=" + String(temperature, 2);

    http.begin(url);
    int responseCode = http.GET();

    if (responseCode > 0)
    {
      String response = http.getString();
      Serial.print("ThingSpeak response: ");
      Serial.println(response);

      if (response.toInt() > 0)
        Serial.println("Temperature uploaded successfully!");
```

```
      else
        Serial.println("Upload not accepted. Check API key and channel.");
    }
    else
    {
      Serial.print("HTTP error: ");
      Serial.println(responseCode);
    }
    http.end();
  }
  else
  {
    Serial.println("Wi-Fi disconnected. Reconnecting...");
    connectWiFi();
  }
}

void setup()
{
  Serial.begin(115200);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  analogReadResolution(12);
  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Smart Temp IoT");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");

  Serial.println("\\nSmart Temperature Monitoring System");
  connectWiFi();
  delay(1500);
  lcd.clear();
}

void loop()
{
  float temperature = readTemperature();

  if (isnan(temperature))
  {
    Serial.println("Invalid NTC sensor reading!");
    lcd.clear();
    lcd.print("Sensor Error");
    delay(2000);
    return;
```

```
  }

  String status = controlLEDs(temperature);
  displayLCD(temperature, status);

  Serial.println("----------------------------");
  Serial.print("Temperature: ");
  Serial.print(temperature, 2);
  Serial.println(" C");
  Serial.print("Status: ");
  Serial.println(status);
  Serial.println("----------------------------");

  if (millis() - previousUpload >= uploadInterval)
  {
    uploadToThingSpeak(temperature);
    previousUpload = millis();
  }

  delay(1000);
}
```

10. CONCLUSION

The Smart Temperature Monitoring System was upgraded into a cloud-connected IoT application using an ESP32 simulated in Wokwi and the ThingSpeak platform. The system reads an NTC sensor, displays temperature and status on an I2C LCD, activates the appropriate LED, and uploads readings to ThingSpeak over Wi-Fi. The successful upload response and Field 1 chart verified device-to-cloud communication and remote temperature visualization.